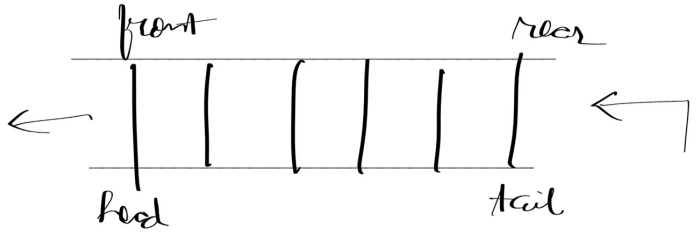
Day 08 ⇒ Queue Interface ⇒

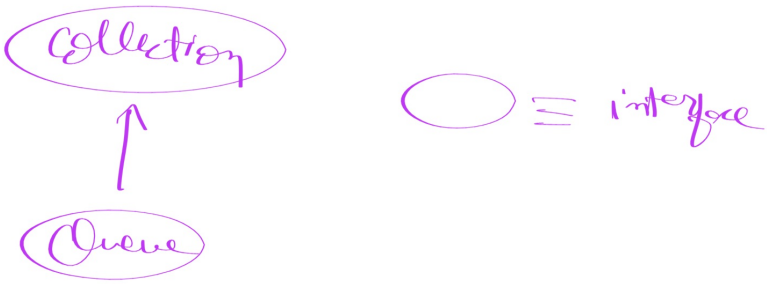
↳ represents a collection designed for holding elements prior to processing. It follows the FIFO (First-In-First-Out) principle, meaning elements are processed in order they were added

Queue



nulls ✗

duplicates ✓



All methods of Collection interface are here in Queue

But some extra methods are also present which are

offer(Ee) ⇒ insert an element from tail side

↳ return boolean

↓

return false if queue is full

we can also use add(Ee) but it will through exception.

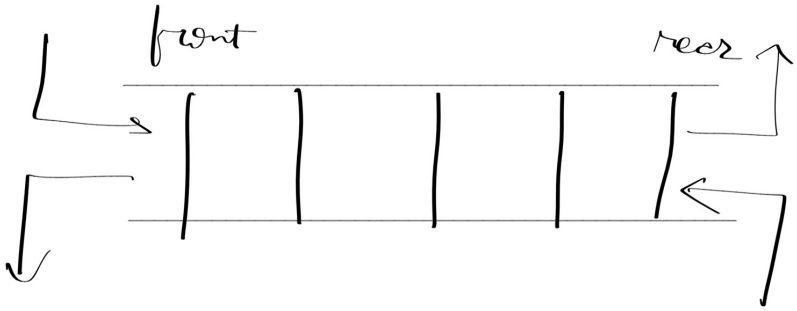
poll() → retrieves and removes the head of queue.
Return null if queue is empty

remove(Ee) will throw exception if queue is empty

peek() → retrieving but does not remove the head element. Return null if empty.

Deque (Double Ended Queue) → an interface

↳ a linear collection that supports element insertion and removal at both ends. It can function as both a queue (FIFO) and a stack (LIFO)



common implementations including ArrayDeque and LinkedList

methods =>

① addLast → from end

② addFirst → from first

③ offerFirst

④ offerLast

⑤ peekLast

⑥ peekFirst

⑦ getLast
⑧ getFirst
} throw exception if empty

It is recommended not to store null in deque & if in it poll will return null which will lead to confusion.

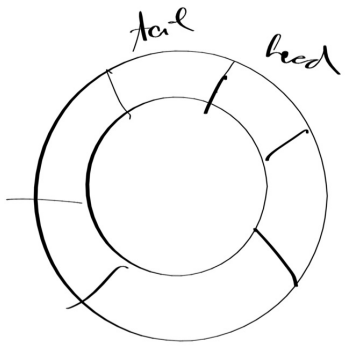
Using ArrayDeque in implementing Deque ⇒

↳ is a resizable array implementation of the deque interface in Java collections framework. It provides a versatile efficient structure for adding and removing elements from both end.

① Dynamic resizing → initial capacity 16 elements that will double when full

② No null elements are allowed.

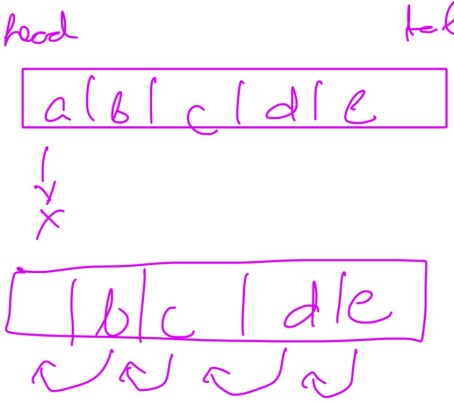
③ Circular array ⇒



no shifting required

& insertion O(1)

where as in case of ArrayList
↳ also implements array
internally we can not use for deque



→ shifting requires O(n)
∴ insertion require O(n)

∴ ArrayDeque is used for implementing deque

(4) O(1) operation
↳ for add, remove and access operation at both end of deque

ArrayDeque is a class
↳ initial capacity
(16+1)
↳ 1 for holding null element at tail

we can also no. of elements we required initially

Why ArrayDeque Beats LinkedList

(1) Reduced Garbage Collection

(2) Superior Memory Efficiency

(3) Faster Iteration Speed

(4) Better Code Locality